PROJECT#5

RESTAURANT RESERVATION

Presented By:

- Kattiyanee Setabutr
- Pranithi Tungmunkij
- Peerapat Setsuk

01

2/2025

be0

Restaurant Reservation

โจทย์ที่ได้รับ

Project#5: Restaurant Reservation

1. The system shall allow a user to register by specifying the name, telephone number, email, and password.
2. After registration, the user becomes a registered user, and the system shall allow the user to log in to use the system by specifying the email and password. The system shall allow a registered user to log out.
3. After login, the system shall allow the registered user to reserve up to 3 tables by specifying the date and the preferred restaurant. The restaurant list is also provided to the user. A restaurant information includes the name, address, telephone number, and open-close time.
4. The system shall allow the registered user to view his/her restaurant reservation.
5. The system shall allow the registered user to edit his/her restaurant reservation.
6. The system shall allow the registered user to delete his/her restaurant reservation.
7. The system shall allow the admin to view any restaurant reservation.
8. The system shall allow the admin to edit any restaurant reservation.
9. The system shall allow the admin to delete any restaurant reservation.

โค้ดโปรแกรมที่ได้แก้ไขไปจากของเดิม (CODE ASSIGNMENT 7)

02

db.js

ASSIGNMENT 7

```javascript
const mongoose = require('mongoose');

const connectDB = async () => {
    mongoose.set('strictQuery', true);
    const conn = await mongoose.connect(process.env.MONGO_URI);

    console.log(`MongoDB Connected: ${conn.connection.host}`);
}

module.exports = connectDB;
```

RESTAURANT RESERVATION

```javascript
import mongoose from 'mongoose';
import dotenv from 'dotenv';
dotenv.config();

const connectionString = process.env.DATABASE_URL;

if (!connectionString) {
    throw new Error('DATABASE_URL environment variable is not defined');
}

export async function connectDB() {
    const conn = await mongoose.connect(connectionString);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
}

export default connectDB;
```

controllers/auth.js

ASSIGNMENT 7

```javascript
// @desc    Register user
// @route   POST /api/v1/auth/register
// @access  Public
exports.register = async (req, res, next) => {
    try {
        const {name, email, password, role} = req.body;

        // Create user
        const user = await User.create ({
            name,
            email,
            password,
            role
        });
        // const token = user.getSignedJwtToken();
        // res.status(200).json({success: true, token});
        sendTokenResponse(user, 200, res);
    } catch(err) {
        res.status(400).json({success: false});
        console.log(err.stack);
    }
};
```

RESTAURANT RESERVATION

```javascript
// @desc    Register user
// @route   POST /api/auth/register
// @access  Public
export const register = async (req, res) => {
    try {
        const { name, telephone_number, email, password, adminSecret } = req.body;

        let role = 'user';
        if (adminSecret && adminSecret === process.env.ADMIN_SECRET) {
            role = 'admin';
        }

        const user = await User.create({
            name,
            telephone_number,
            email,
            password,
            role
        });

        sendTokenResponse(user, 201, res);
    } catch (err) {
        res.status(400).json({success: false});
    }
};
```

controllers/auth.js

ASSIGNMENT 7

```javascript
// @desc    Login user
// @route   POST /api/v1/auth/login
// @access  Public
exports.login = async (req, res, next) => {
    const {email, password} = req.body;

    // Validate email & password
    if(!email || !password) {
        return res.status(400).json({
            success: false,
            msg: 'Please provide an email and password'
        });
    }

    // Check for user
    const user = await User.findOne({email}).select('+password');

    if(!user) {
        return res.status(400).json({
            success: false,
            msg: 'Invalid credentials'
        });
    }

    // Check if password matches
    const isMatch = await user.matchPassword(password);

    if(!isMatch) {
        return res.status(401).json({
            success: false,
            msg: 'Invalid credentials'
        });
    }

    // Create token
    // const token = user.getSignedJwtToken();
    // res.status(200).json({success: true, token});
    sendTokenResponse(user, 200, res);
};
```

RESTAURANT RESERVATION

```javascript
// @desc    Login user
// @route   POST /api/auth/login
// @access  Public
export const login = async (req, res) => {
    try {
        const { email, password } = req.body;

        if (!email || !password) {
            return res.status(400).json({
                success: false,
                message: 'Please provide an email and password'
            });
        }

        const user = await User.findOne({ email }).select('+password');

        if (!user) {
            return res.status(400).json({
                success: false,
                message: 'Invalid credentials'
            });
        }

        const isMatch = await user.matchPassword(password);

        if (!isMatch) {
            return res.status(400).json({
                success: false,
                message: 'Invalid credentials'
            });
        }

        sendTokenResponse(user, 200, res);
    } catch (err) {
        res.status(500).json({success: false});
    }
};
```

controllers/auth.js

RESTAURANT RESERVATION

```javascript
// @desc    Log user out / clear cookie
// @route   GET /api/auth/logout
// @access  Private
export const logout = async (req, res) => {
    res.cookie('token', 'none', {
        expires: new Date(Date.now() + 10 * 1000),
        httpOnly: true
    });

    res.status(200).json({
        success: true,
        data: {}
    });
};
```

controllers/auth.js

ASSIGNMENT 7

```javascript
// Get token from model, create cookie and send response
const sendTokenResponse = (user, statusCode, res) => {
    // Create token
    const token = user.getSignedJwtToken();

    const options = {
        expires:new Date(Date.now()+process.env.JWT_COOKIE_EXPIRE*24*60*60*1000),
        httpOnly: true
    };

    if(process.env.NODE_ENV === 'production') {
        options.secure = true;
    }
    res.status(statusCode).cookie('token', token, options).json({
        success: true,
        token
    })
}

// @desc    Get current Logged in user
// @route   POST /api/v1/auth/me
// @access  Private
exports.getMe = async (req, res, next) => {
    const user = await User.findById(req.user.id);
    res.status(200).json({
        success: true,
        data: user
    });
}
```

RESTAURANT RESERVATION

```javascript
// @desc    Get current logged in user
// @route   GET /api/auth/me
// @access  Private
export const getMe = async (req, res) => {
    try {
        const user = await User.findById(req.user.id);
        res.status(200).json({ success: true, data: user });
    } catch (err) {
        res.status(500).json({ success: false, message: 'Cannot get user info' });
    }
};

const sendTokenResponse = (user, statusCode, res) => {
    const token = user.getSignedJwtToken();

    const options = {
        expires: new Date(Date.now() + process.env.JWT_COOKIE_EXPIRE * 24 * 60 * 60 * 1000),
        httpOnly: true
    };

    if (process.env.NODE_ENV === 'production') {
        options.secure = true;
    }

    res.status(statusCode).cookie('token', token, options).json({ success: true, token });
};

```

controllers/reservations.js

ASSIGNMENT 7

```javascript
// @desc    Get all appointments
// @route   GET /api/v1/appointments
// @access  Public
exports.getAppointments = async (req, res, next) => {
    let query;

    // General users can see only their appointments!
    if(req.user.role !== 'admin') {
        query = Appointment.find({user: req.user.id}).populate({
            path: 'hospital',
            select: 'name province tel'
        });
    } else { // If you are an admin, you can see all!
        query = Appointment.find().populate({
            path: 'hospital',
            select: 'name province tel'
        });
    }

    try {
        const appointments = await query;

        res.status(200).json ({
            success: true,
            count: appointments.length,
            data: appointments
        });
    } catch(error) {
        console.log(error);
        return res.status(500).json({
            success: false,
            message: "Cannot find Appointment"
        });
    }
}
```

RESTAURANT RESERVATION

```javascript
// @desc    Get all reservations
// @route   GET /api/reservations
// @access  Private
export const getReservations = async (req, res) => {
    try {
        let query = {};

        if (req.params.restaurantId) {
            query.restaurant_id = req.params.restaurantId;
        }

        if (req.user.role !== 'admin') {
            query.user = req.user.id;
        }

        const reservations = await Reservation.find(query)
            .populate('user', 'name email')
            .populate('restaurant_id', 'name address');

        res.status(200).json({
            success: true,
            count: reservations.length,
            data: reservations
        });
    } catch (err) {
        res.status(500).json({
            success: false,
            message: 'Cannot find reservations'
        });
    }
};
```

controllers/reservations.js

ASSIGNMENT 7

```javascript
// @desc    Get single appointment
// @route   GET /api/v1/appointments/:id
// @access  Public
exports.getAppointment = async (req, res, next) => {
    try {
        const appointment = await Appointment.findById(req.params.id).populate({
            path: 'hospital',
            select: 'name description tel'
        });

        if(!appointment) {
            return res.status(404).json({
                success:false,
                message:`No appointment with the id of ${req.params.id}`
            });
        }

        res.status(200).json({
            success:true,
            data:appointment
        });
    } catch(error) {
        console.log(error);
        return res.status(500).json({
            success:false,
            message:"Cannot find Appointment"
        });
    }
}
```

RESTAURANT RESERVATION

```javascript
// @desc    Get single reservation
// @route   GET /api/reservations/:id
// @access  Private
export const getReservation = async (req, res) => {
    try {
        const reservation = await Reservation.findById(req.params.id)
            .populate('user', 'name email')
            .populate('restaurant_id', 'name address');

        if (!reservation) {
            return res.status(404).json({
                success: false,
                message: `No reservation with the id of ${req.params.id}`
            });
        }

        if (reservation.user._id.toString() !== req.user.id.toString() && req.user.role !== 'admin') {
            return res.status(401).json({
                success: false,
                message: `User ${req.user.id} is not authorized to view this reservation`
            });
        }

        res.status(200).json({
            success: true,
            data: reservation
        });
    } catch (err) {
        res.status(500).json({
            success: false,
            message: 'Cannot find reservation'
        });
    }
};
```

middleware/auth.js

ASSIGNMENT 7

```javascript
exports.protect = async(req, res, next) => {
    let token;

    if(req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
        token = req.headers.authorization.split(' ')[1];
    }

    // Make sure token exists
    if(!token) {
        return res.status(401).json({
            success: false,
            message: 'Not authorize to access this route'
        });
    }

    try {
        // Verify token
        const decoded = jwt.verify(token, process.env.JWT_SECRET);

        console.log(decoded);

        req.user = await User.findById(decoded.id);

        next();
    } catch(err) {
        console.log(err.stack);
        return res.status(401).json({
            success: false,
            message: 'Not authorize to access this route'
        });
    }
}
```

RESTAURANT RESERVATION

```javascript
export const protect = async (req, res, next) => {
    let token;

    if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
        token = req.headers.authorization.split(' ')[1];
    } else if (req.cookies.token) {
        token = req.cookies.token;
    }

    if (!token || token === 'null') {
        return res.status(401).json({
            success: false,
            message: 'Not authorized to access this route'
        });
    }

    try {
        const decoded = jwt.verify(token, process.env.JWT_SECRET);
        req.user = await User.findById(decoded.id);
        next();
    } catch (err) {
        return res.status(401).json({
            success: false,
            message: 'Not authorized to access this route'
        });
    }
};
```

models/Reservation.js

ASSIGNMENT 7

```javascript
const AppointmentSchema = new mongoose.Schema({
    apptDate: {
        type: Date,
        required: true
    },
    user: {
        type: mongoose.Schema.ObjectId,
        ref: 'User',
        required: true
    },
    hospital: {
        type: mongoose.Schema.ObjectId,
        ref: 'Hospital',
        required: true
    },
    createdAt: {
        type: Date,
        default: Date.now
    }
});
```

RESTAURANT RESERVATION

```javascript
const ReservationSchema = new mongoose.Schema({
    user: {
        type: mongoose.Schema.ObjectId,
        ref: 'User',
        required: true
    },
    restaurant_id: {
        type: mongoose.Schema.ObjectId,
        ref: 'Restaurant',
        required: true
    },
    restaurant_name: {
        type: String
    },
    reservation_date: {
        type: Date,
        required: [true, 'Please add a reservation date']
    },
    quantity: {
        type: Number,
        default: 1,
        min: [1, 'Quantity must be at least 1']
    },
    table: {
        type: mongoose.Schema.ObjectId,
        ref: 'Table'
    },
    createdAt: {
        type: Date,
        default: Date.now
    }
});
```

models/Restaurant.js

ASSIGNMENT 7

```javascript
const HospitalSchema = new mongoose.Schema({
    name: {
        type: String,
        required: [true,'Please add a name'],
        unique: true,
        trim: true,
        maxlength: [50,'Name can not be more than 50 characters']
    },
    address: {
        type: String,
        required: [true, 'Please add an address']
    },
    district: {
        type: String,
        required: [true, 'Please add a district']
    },
    province: {
        type: String,
        required: [true, 'Please add a province']
    },
    postalcode: {
        type: String,
        required: [true, 'Please add a postalcode'],
        maxlength: [5, 'Postal Code can not be more than 5 digits']
    },
    tel: {
        type: String
    },
    region: {
        type: String,
        required: [true, 'Please add a region']
    }
}, {
    toJSON: {virtuals: true},
    toObject:{virtuals: true}
});

// Reverse populate with virtuals
HospitalSchema.virtual('appointments', {
    ref: 'Appointment',
    localField: '_id',
    foreignField: 'hospital',
    justOne: false
});
```

RESTAURANT RESERVATION

```javascript
const RestaurantSchema = new mongoose.Schema({
    name: {
        type: String,
        required: [true, 'Please add a name'],
        unique: true,
        trim: true,
        maxlength: [50, 'Name cannot be more than 50 characters']
    },
    address: {
        type: String,
        required: [true, 'Please add an address']
    },
    telephone_number: {
        type: String
    },
    open_time: {
        type: Date
    },
    close_time: {
        type: Date
    },
    createdAt: {
        type: Date,
        default: Date.now
    }
}, {
    toJSON: {virtuals: true},
    toObject: {virtuals: true}
});

// Virtual populate reservations
RestaurantSchema.virtual('reservations', {
    ref: 'Reservation',
    localField: '_id',
    foreignField: 'restaurant_id',
    justOne: false
});

// Virtual populate tables
RestaurantSchema.virtual('tables', {
    ref: 'Table',
    localField: '_id',
    foreignField: 'restaurant_id',
    justOne: false
});
```

models/User.js

ASSIGNMENT 7

```js
const UserSchema = new mongoose.Schema({
    name: {
        type:String,
        required:[true, 'Please add a name']
    },
    email: {
        type: String,
        required:[true, 'Please add an email'],
        unique: true,
        match: [
            /^(([^<>()[\]\\.,;:\s@"]+(\.[^<>()[\]\\.,;:\s@"]+)*)|(".+"))@((\[[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\])|(([a-zA-Z\-0-9]+\.)+[a-zA-Z]{2,}))$/,
            'Please add a valid email'
        ]
    },
    role: {
        type:String,
        enum: ['user', 'admin'],
        default: 'user'
    },
    password: {
        type:String,
        required:[true, 'Please add a password']
    ,
        minlength: 6,
        select: false
    },
    resetPasswordToken: String,
    resetPasswordExpire: Date,
    createdAt: {
        type: Date,
        default:Date.now
    }
});

// Encrypt password using bcrypt
UserSchema.pre('save', async function(next) {
    const salt = await bcrypt.genSalt(10);
    this.password = await bcrypt.hash(this.password, salt);
});
```

RESTAURANT RESERVATION

```js
const UserSchema = new mongoose.Schema({
    name: {
        type: String,
        required: [true, 'Please add a name']
    },
    telephone_number: {
        type: String,
        required: [true, 'Please add a telephone number'],
        unique: true,
        match: [
            /^[0-9+\-]{9,15}$/,
            'Please add a valid telephone number'
        ]
    },
    email: {
        type: String,
        required: [true, 'Please add an email'],
        unique: true,
        match: [
            /^(([^<>()[\]\\.,;:\s@"]+(\.[^<>()[\]\\.,;:\s@"]+)*)|(".+"))@((\[[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\])|(([a-zA-Z\-0-9]+\.)+[a-zA-Z]{2,}))$/,
            'Please add a valid email'
        ]
    },
    role: {
        type: String,
        enum: ['user', 'admin'],
        default: 'user'
    },
    password: {
        type: String,
        required: [true, 'Please add a password'],
        minlength: 6,
        select: false
    },
    createdAt: {
        type: Date,
        default: Date.now
    }
});

// Encrypt password using bcrypt before saving
UserSchema.pre('save', async function (next) {
    if (!this.isModified('password')) return next();
    const salt = await bcrypt.genSalt(10);
    this.password = await bcrypt.hash(this.password, salt);
});
```

routes/auth.js

ASSIGNMENT 7

```
const express = require('express');
const {register, login, getMe} = require('../controllers/auth');

const router = express.Router();

const {protect} = require('../middleware/auth');
router.post('/register', register);
router.post('/login', login);
router.get('/me', protect, getMe);

module.exports = router;
```

RESTAURANT RESERVATION

```
import express from 'express';
import { register, login, logout, getMe } from '../controllers/auth.js';
import { protect } from '../middleware/auth.js';

const router = express.Router();

router.post('/register', register);
router.post('/login', login);
router.get('/logout', protect, logout);
router.get('/me', protect, getMe);

export default router;
```

routes/reservations.js

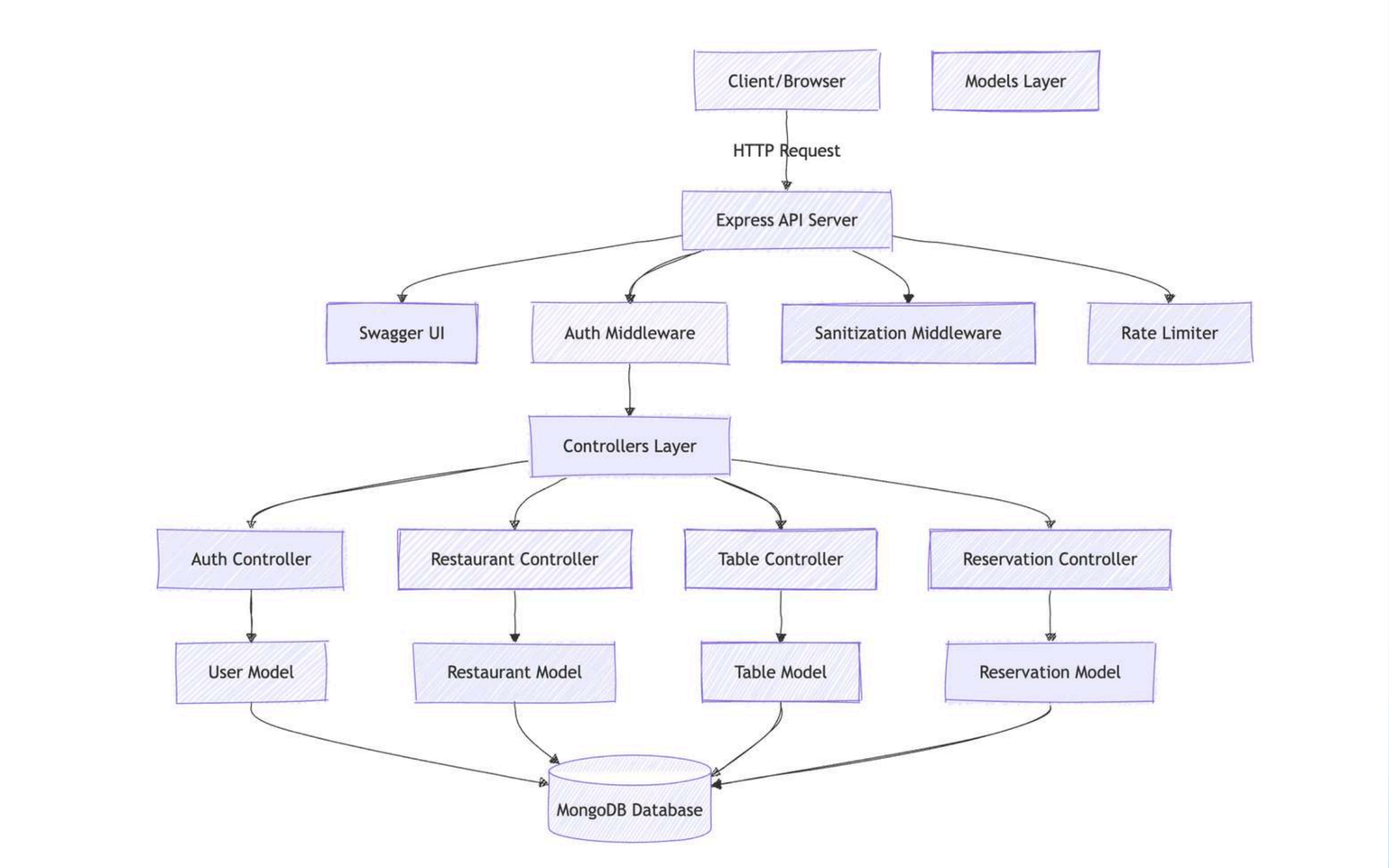
ASSIGNMENT 7

```javascript
const express = require('express');
const {getAppointments, getAppointment, addAppointment, updateAppointment, deleteAppointment} = require('../controllers/appointments');

const router = express.Router({mergeParams:true});

const {protect, authorize} = require('../middleware/auth');

router.route('/')
    .get(protect, getAppointments)
    .post(protect, authorize('admin', 'user'), addAppointment);
router.route('/:id')
    .get(protect, getAppointment)
    .put(protect, authorize('admin', 'user'), updateAppointment)
    .delete(protect, authorize('admin', 'user'), deleteAppointment);

module.exports = router;
```

RESTAURANT RESERVATION

```javascript
import express from 'express';
import { getReservations, getReservation, addReservation, updateReservation, deleteReservation } from '../controllers/reservations.js';
import { protect } from '../middleware/auth.js';

const router = express.Router({ mergeParams: true });

router.use(protect);

router.route('/').get(getReservations).post(addReservation);
router.route('/:id').get(getReservation).put(updateReservation).delete(deleteReservation);

export default router;
```

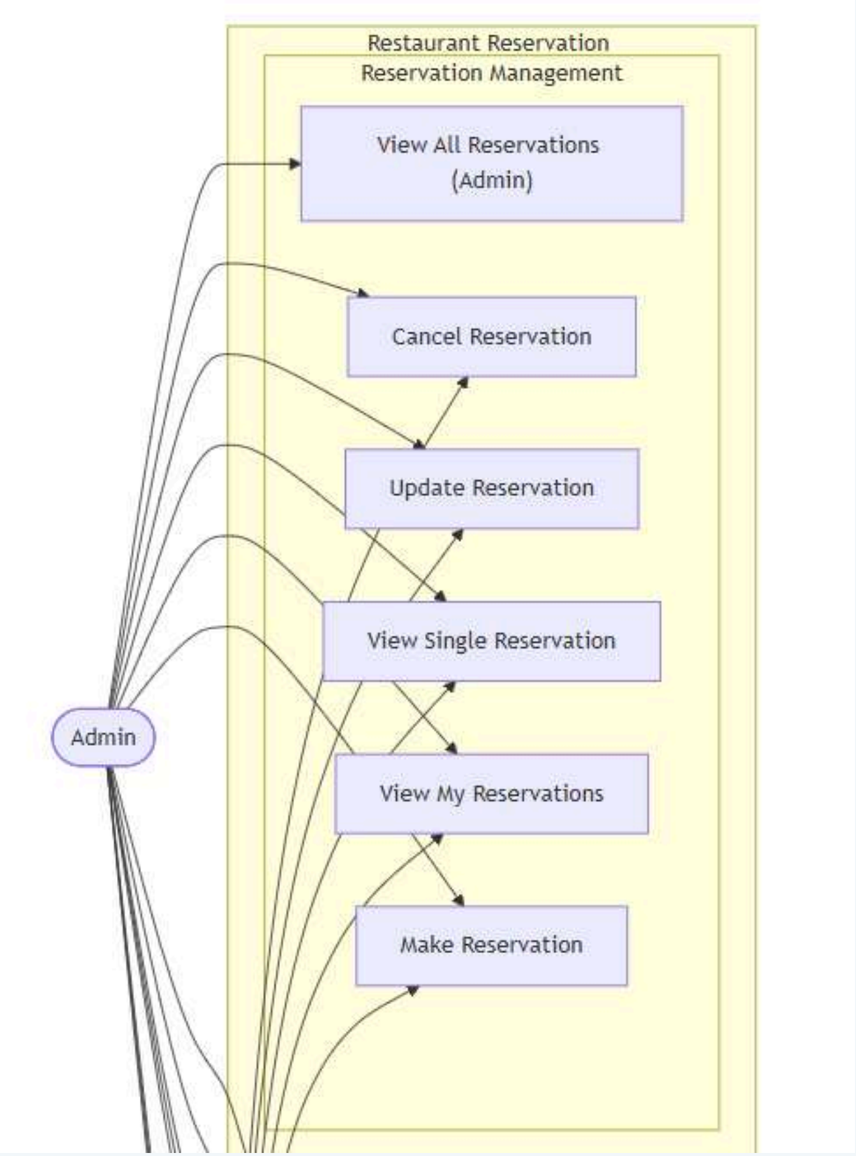
SYSTEM OVERVIEW



03

USE CASE DIAGRAM

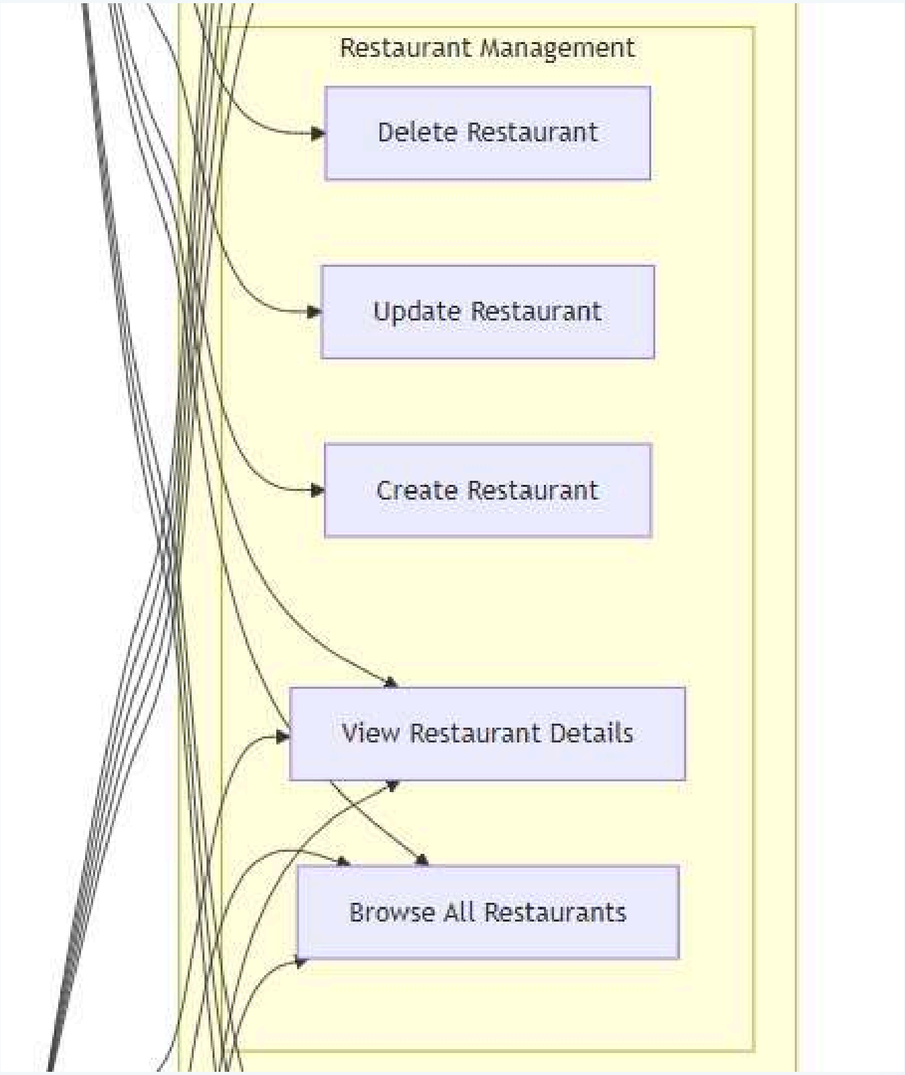
Restaurant Reservation beO 2/2025



03

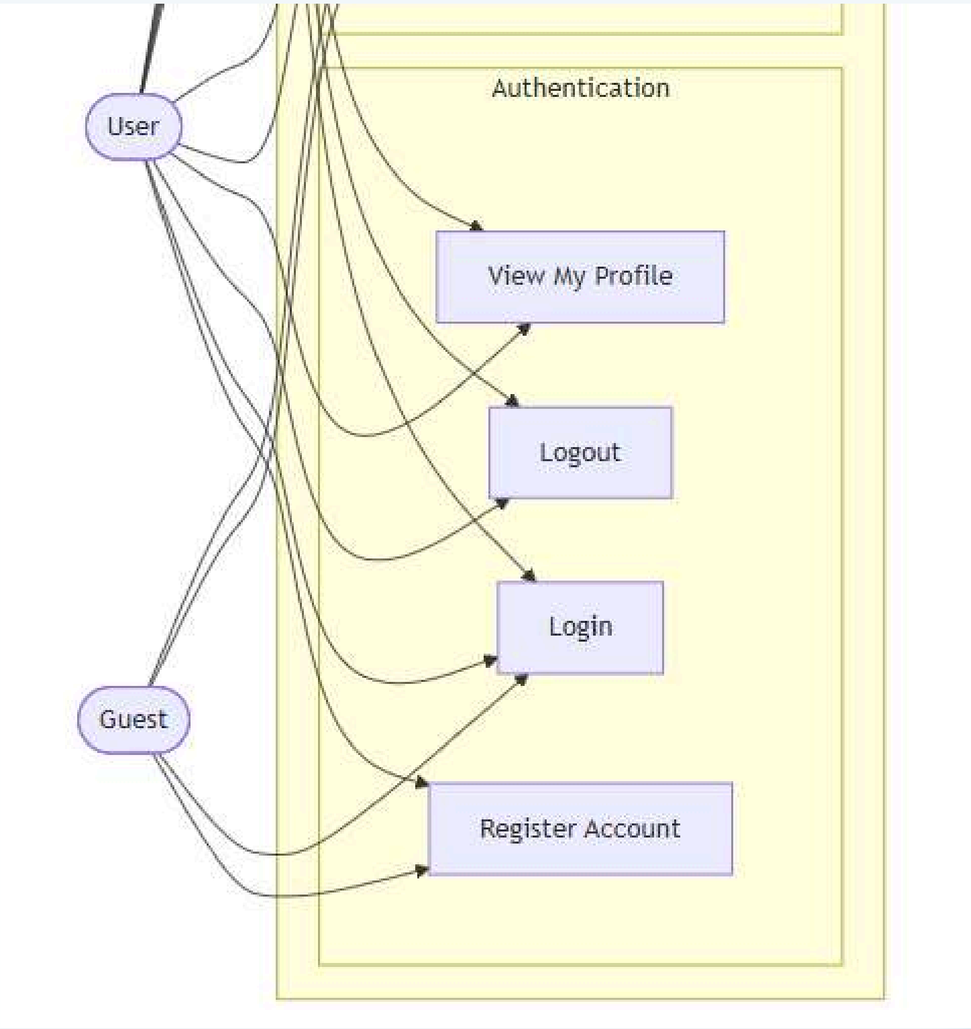
USE CASE DIAGRAM

Restaurant Reservation beO 2/2025



03

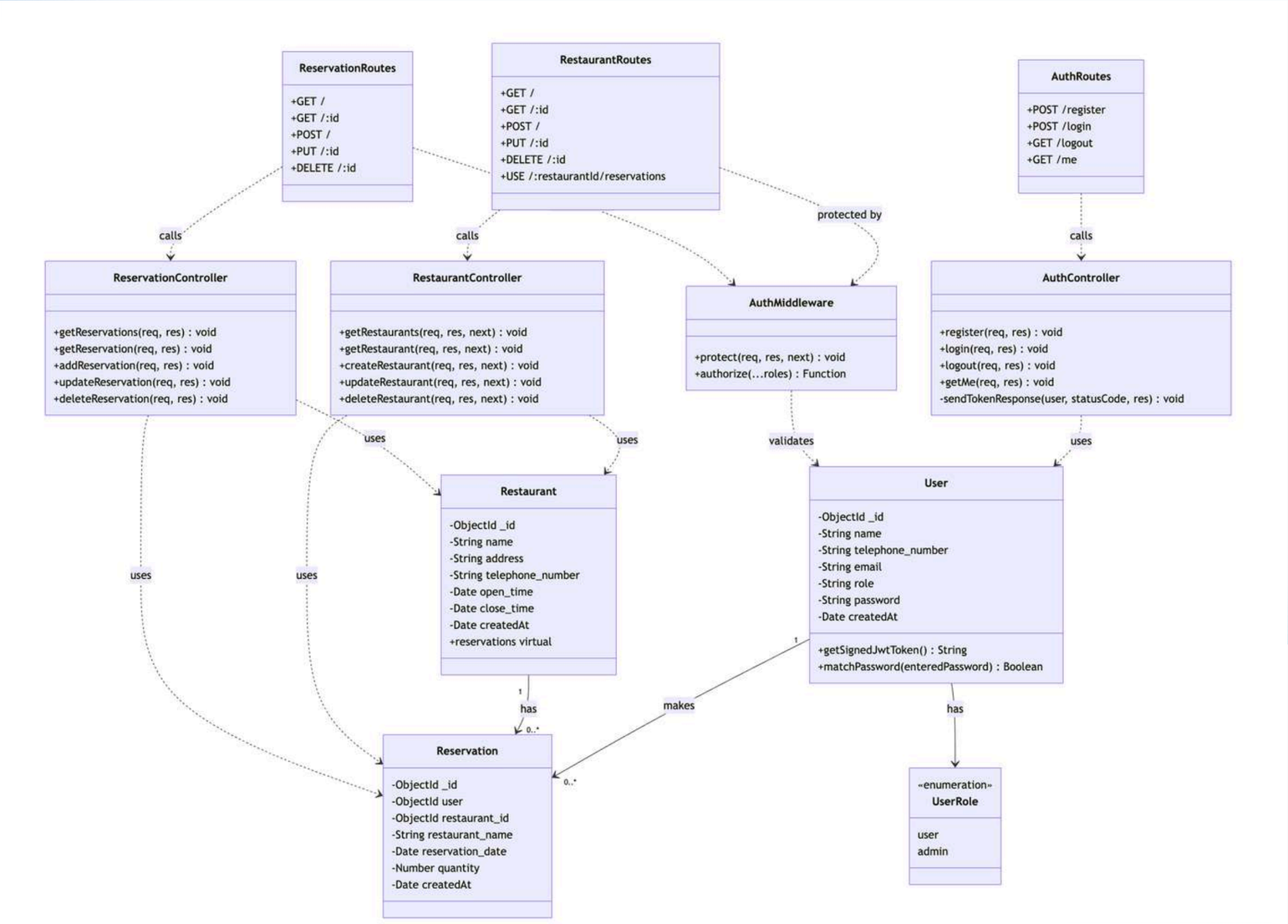
USE CASE DIAGRAM



03

CLASS DIAGRAM (UML PROFILE)

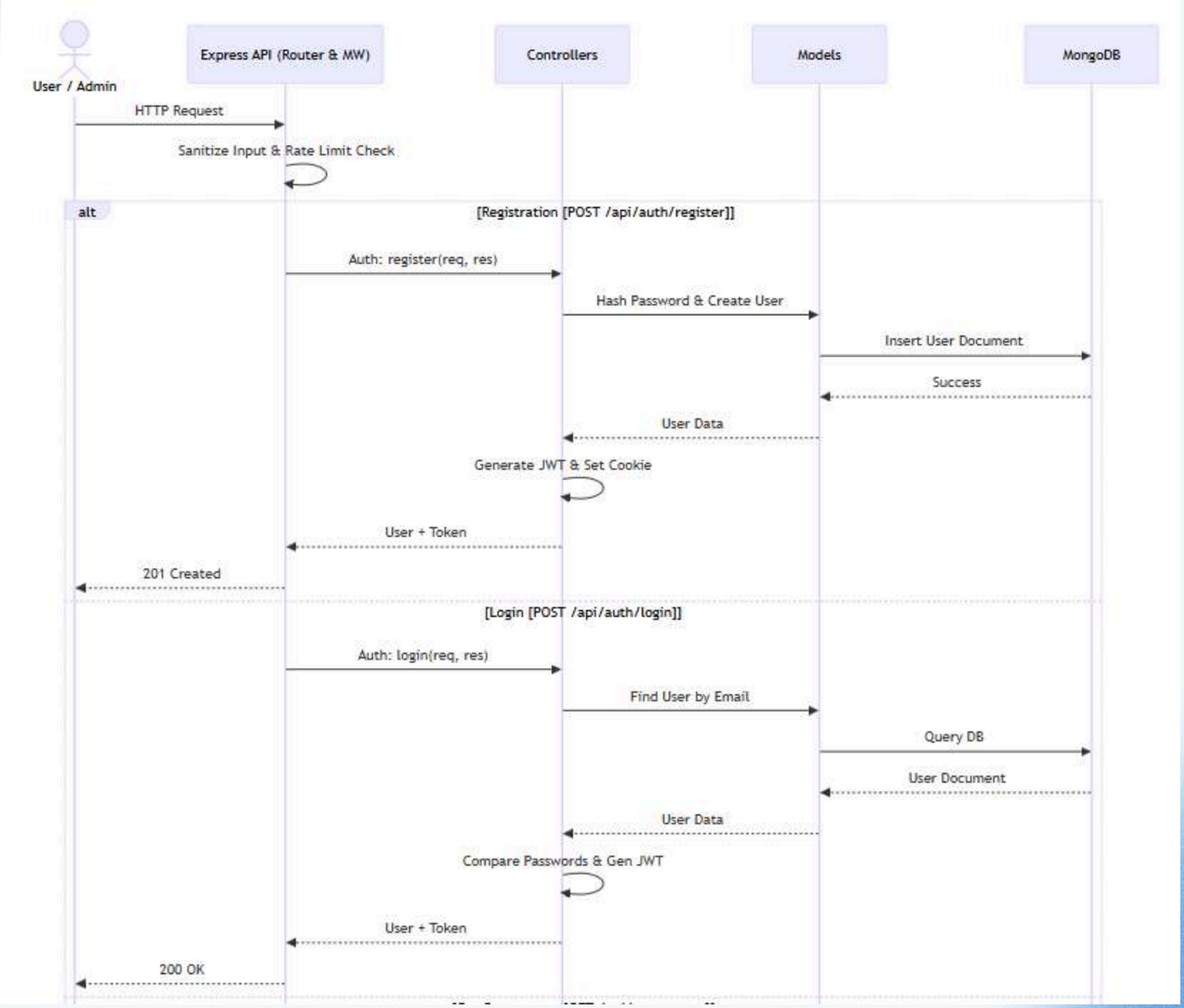
Restaurant Reservation beO 2/2025



03

SEQUENCE DIAGRAM

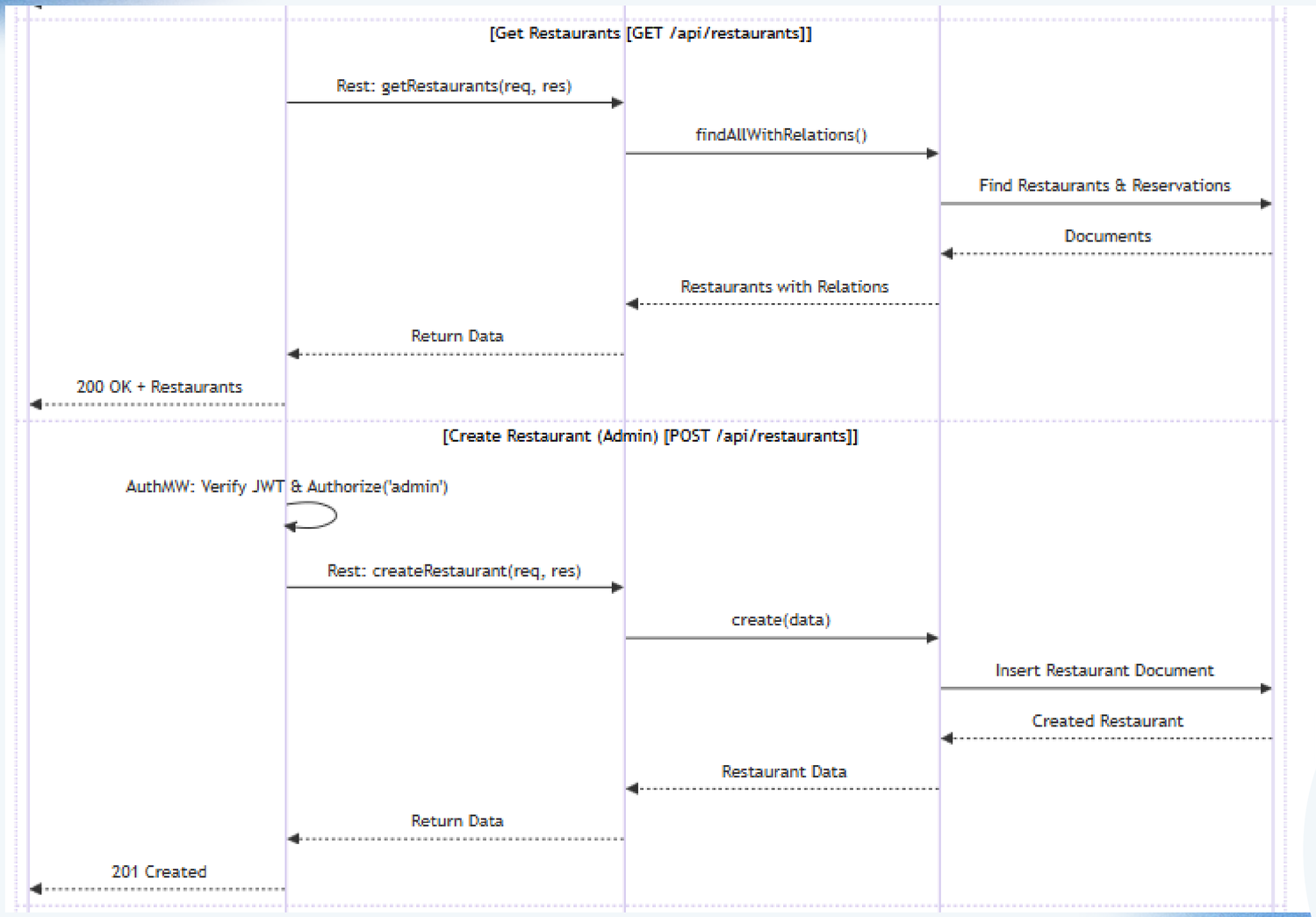
Restaurant Reservation beO 2/2025



03

SEQUENCE DIAGRAM

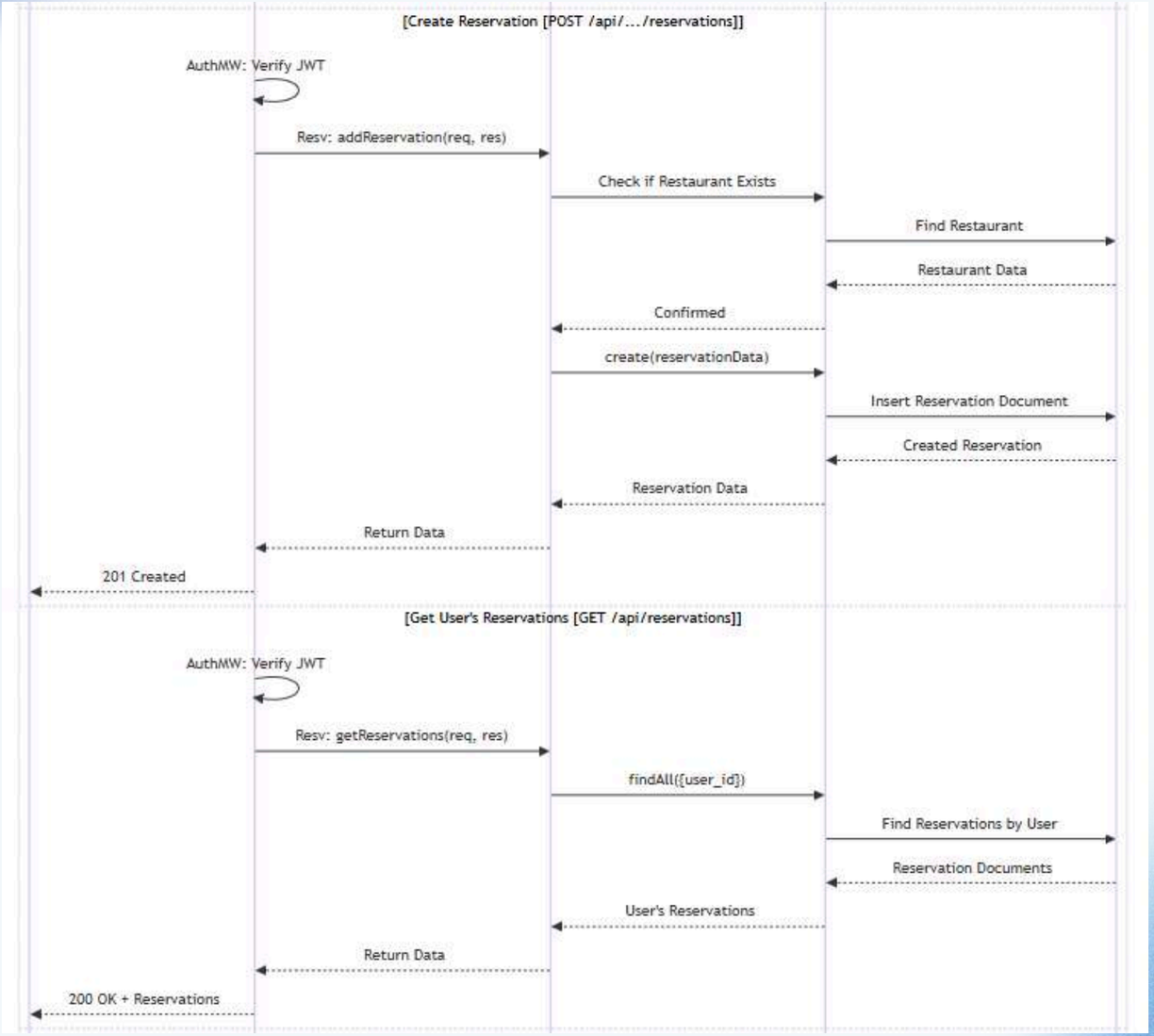
Restaurant Reservation beO 2/2025



03

SEQUENCE DIAGRAM

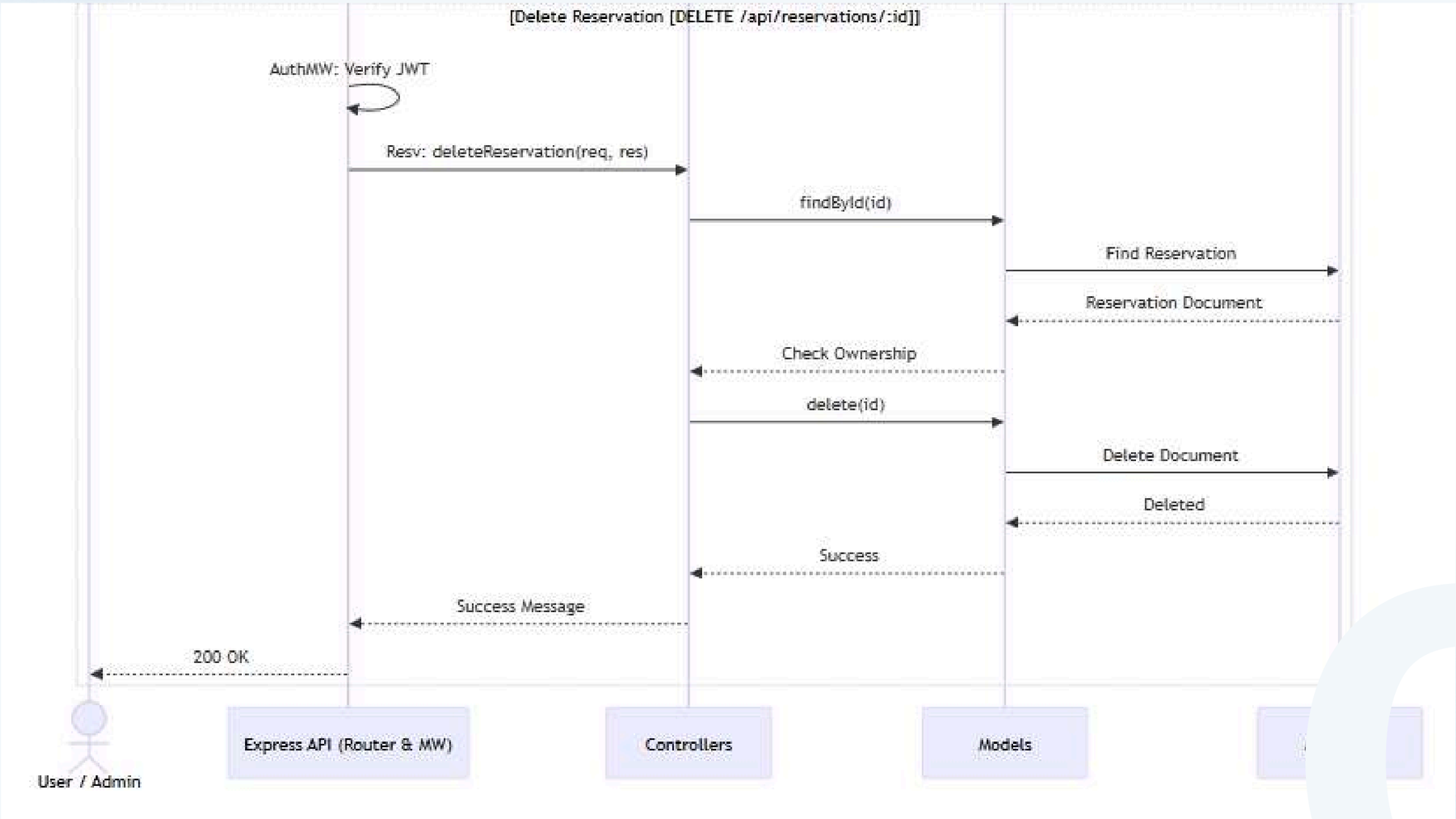
Restaurant Reservation beO 2/2025



03

SEQUENCE DIAGRAM

Restaurant Reservation beO 2/2025



03

04

การแบ่งงานในทีม

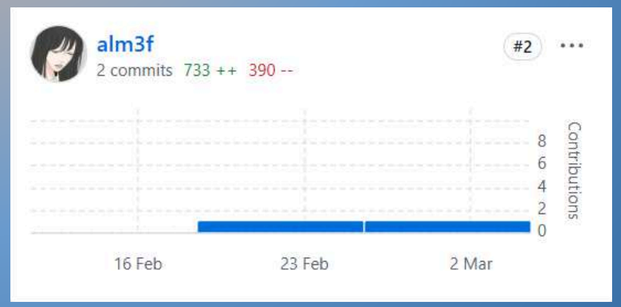
(GITHUB >> CONTRIBUTORS)

PEERAPAT SETSUK

Logic Restaurant and Reservation

KATTIYANEE SETABUTR

feature ใน security และซัพพอร์ตด้าน
ต่าง ๆ (เช่น migration mongoose)



PRANITHI TUNGMUNKIJ

auth

05

LIST ของ ADDITIONAL REQUIREMENTS

HTTPS://YOUTU.BE/D6H_-ELNCTC

Open Hours Validation (controllers/reservations.js)

```
1  if (restaurant.open_time && restaurant.close_time) {
2        const resTotalMinutes =
3          resDate.getUTCHours() * 60 + resDate.getUTCMinutes();
4
5        const [openHH, openMM] = restaurant.open_time.split(":").map(Number);
6        const [closeHH, closeMM] = restaurant.close_time.split(":").map(Number);
7        const openMinutes = openHH * 60 + openMM;
8        const closeMinutes = closeHH * 60 + closeMM;
9
10       if (resTotalMinutes < openMinutes || resTotalMinutes > closeMinutes) {
11         return res.status(400).json({
12           success: false,
13           message: `Restaurant is open from ${restaurant.open_time} to ${restaurant.close_time}`,
14         });
15       }
16     }
```

บรรทัดที่ 105 205 ใน controllers/reservations.js

No Duplicate Same-Day Reservation (controllers/reservations.js)

```
const sameDayStart = new Date(resDate);
    sameDayStart.setUTCHours(0, 0, 0, 0);
    const sameDayEnd = new Date(resDate);
    sameDayEnd.setUTCHours(23, 59, 59, 999);

    const duplicate = await Reservation.findOne({
      user: req.user.id,
      restaurant_id,
      reservation_date: { $gte: sameDayStart, $lte: sameDayEnd },
    });

    if (duplicate) {
      return res.status(400).json({
        success: false,
        message:
          "You already have a reservation at this restaurant on this date",
      });
    }
```

บรรทัดที่ 133 223 ใน controllers/reservations.js

1-Hour Cancellation Lock (controllers/reservations.js)

```
if (req.user.role !== "admin") {
      const timeDiff = new Date(reservation.reservation_date) - new Date();
      if (timeDiff < 60 * 60 * 1000) {
        return res.status(400).json({
          success: false,
          message:
            "Cannot cancel reservation within 1 hour of reservation time",
        });
      }
    }
```

บรรทัดที่ 283 ใน controllers/reservations.js

Admin Registration via Secret (controllers/auth.js)

```
let role = 'user';
        if (adminSecret && adminSecret === process.env.ADMIN_SECRET) {
            role = 'admin';
        }
```

บรรทัดที่ 10 ใน controllers/auth.js

LINK TO PROJECT GITHUB REPO

https://github.com/2110503-CEDT68/be-project-68-be0

06